

МОДУЛЬ 4 · УРОК 4.2

# Цикл **for**

Летим к звёздам: считаем звёзды в небе и копим очки за весь полёт.

**C++ с нуля** · 45 минут

Когда заранее знаешь, **сколько**  
**раз** повторить — выбирай **for** .

Сегодня посчитаем звёзды и соберём сумму очков экспедиции.

# Вспомним прошлый урок

- Цикл `while` повторяет тело, пока верно условие
- У цикла есть счётчик: старт, условие, изменение
- Счётчик должен меняться — иначе бесконечный цикл

`for` — это тот же цикл, но все три части записаны **в одну строку**.

# План урока

- 1 Чем `for` удобнее `while`
- 2 Три части `for` в одной строке
- 3 Перебор диапазона: от 1 до N
- 4 Накопление суммы — копилка очков
- 5 **Практика в классе** — звёзды и очки полёта
- 6 **Домашнее задание**

# while и for — это одно и то же

В `while` три части цикла разбросаны. В `for` — собраны вместе.

## while

```
main.cpp

int i = 1;
while (i <= 5) {
    std::cout << i;
    i = i + 1;
}
```

## for

```
main.cpp

for (int i = 1; i <= 5; i++) {
    std::cout << i;
}
```

Делают одно и то же. `for` короче — три части в одной строке.

# Три части for

main.cpp

```
for (int i = 1; i <= 5; i++) {  
    std::cout << i << " ";  
}
```

- `int i = 1` — **старт**: с чего начать счётчик
- `i <= 5` — **условие**: пока повторяем
- `i++` — **шаг**: увеличить счётчик на 1

`i++` — это короткая запись для `i = i + 1`.

# Что такое `i++`

`i++` означает «увеличь `i` на единицу». Самый частый шаг в `for`.

## Длинно

```
i = i + 1;
```

## Коротко

```
i++
```

Обе записи делают одно и то же. В `for` почти всегда пишут `i++`.

# Как for крутится · шаг за шагом

Следим за `i` в `for (int i = 1; i <= 4; i++)` :

| Шаг | <code>i</code> | <code>i &lt;= 4 ?</code> | Вывод |
|-----|----------------|--------------------------|-------|
| 1   | 1              | да                       | 1     |
| 2   | 2              | да                       | 2     |
| 3   | 3              | да                       | 3     |
| 4   | 4              | да                       | 4     |
| 5   | 5              | нет — стоп               | —     |

Порядок: проверили условие → выполнили тело → сделали `i++` → снова проверили.

# Перебор диапазона



```
main.cpp

for (int i = 1; i <= 10; i++) {
    std::cout << "Звезда №" << i << "\n";
}
```

Один цикл — и программа сама пронумерует все 10 звёзд.

Хочешь до 100? Поменяй только 10 на 100 . Цикл не вырастет ни на строчку.

# Накопление суммы · копилка

Запускаем «копилку» `sum = 0` и на каждом шаге добавляем в неё.

```
main.cpp

int sum = 0;
for (int i = 1; i <= 5; i++) {
    sum = sum + i;
}
std::cout << "Очки: " << sum << "\n";
```

`sum = sum + i` — «ВОЗЬМИ КОПИЛКУ, добавь `i`, положи обратно».

# Как растёт копилка · по шагам

Считаем сумму `1 + 2 + 3 + 4 + 5` :

| Шаг | <code>i</code> | <code>sum</code> было | <code>sum</code> стало |
|-----|----------------|-----------------------|------------------------|
| 1   | 1              | 0                     | 1                      |
| 2   | 2              | 1                     | 3                      |
| 3   | 3              | 3                     | 6                      |
| 4   | 4              | 6                     | 10                     |
| 5   | 5              | 10                    | 15                     |

В конце `sum = 15` — программа сложила все числа сама.

# Частые ошибки



## Забыл `i++`

Без шага счётчик стоит на месте — бесконечный цикл.



## Копилка внутри

`int sum = 0;` **внутри** цикла — обнуляется каждый шаг.



## Лишняя `;`

`for (...);` — тело отвалилось, цикл пустой.

Копилку `sum = 0` заводи **до** цикла, иначе сумма всё время сбрасывается.

# Проверь себя · что выведет программа? 🧠



main.cpp

```
for (int i = 1; i <= 3; i++) {  
    std::cout << i * 2 << " ";  
}
```

Подумай 10 секунд...

Выведет **2 4 6**. Для `i = 1, 2, 3` печатается `i * 2`: то есть 2, 4 и 6.

# Проверь себя · копилка 🧠



main.cpp

```
int sum = 0;
for (int i = 1; i <= 3; i++) {
    sum = sum + i;
}
std::cout << sum;
```

Что окажется в копилке?

Выведет **6**. Копилка собрала `1 + 2 + 3 = 6`.

ПРАКТИКА В КЛАССЕ

# Летим к звёздам

Пишем прямо сейчас: считаем звёзды и копим очки экспедиции.

## ♦ ЗАДАНИЕ

# Задача 1 · Считаем звёзды

Пронумеруй 10 звёзд на небе с помощью `for`:

```
main.cpp

for (int i = 1; i <= 10; i++) {
    std::cout << "Звезда №" << i << "\n";
}
```

- Набери программу целиком (с `#include` и `main`), поменяй число `10` на своё

Время: 6 минут

✦ ЗАДАНИЕ

## Задача 2 · Очки за полёт 🏆

Посчитай сумму очков с 1 по 5 — заведи копилку:

```
main.cpp

int sum = 0;
for (int i = 1; i <= 5; i++) {
    sum = sum + i;
}
std::cout << "Очки: " << sum << "\n";
```

- Запусти и проверь: должно получиться **15**. Поменяй 5 на 10 — сколько станет?

Подсказка: `sum = 0` заводим ДО цикла

♦ ЗАДАНИЕ

# Задача 3 · Таблица топлива

Выведи, сколько топлива нужно на каждую звезду: по 3 единицы.



main.cpp

Звезда 1 — 3 топлива

Звезда 2 — 6 топлива

...

- Подсказка: на каждом шаге печатай  $i * 3$  — для звёзд с 1 по 5

Время: 7 минут

✦ ЗАДАНИЕ

## Задача 4 · Только чётные звёзды ★

★ со звёздочкой

Сложи в копилку только **чётные** числа от 2 до 10: `2 + 4 + 6 + 8 + 10` .

- Подсказка: старт `int i = 2` , шаг `i = i + 2`
- Проверь сам: сумма должна получиться **30**

Кто сложит ещё и нечётные (1+3+5...) – супер!

## Что мы сегодня научились 🦾

- `for` — это `while`, у которого три части в одной строке
- Запись: `for (старт; условие; шаг)`
- `i++` — короткая запись для `i = i + 1`
- Перебор диапазона: от 1 до N одним циклом
- Копилка `sum = sum + i` копит сумму (заводим **до** цикла)

# Домашнее задание

1. Реши рабочий лист 4.2 — звёзды, копилка очков и трассировка `for`.
2. Загляни в шпаргалку 4.2 — устройство `for` под рукой.

Дальше: урок 4.3 — мини-проект «Угадай число»: компьютер загадал, а ты угадываешь!